

HDOC DAY-30 ASSIGNMENT

Menu-Driven Array Operations Using Switch-Case

1. Problem Understanding

The task is to write a menu-driven C program using switch-case to perform three operations on an integer array. The program first reads the array and then repeatedly displays a menu until the user selects Exit.

- Sort the array using Bubble Sort and then search for an element using Binary Search.
- Insert an element at a valid position or delete an element from a valid position.
- Rotate the array elements to the right by k positions.
- Exit the program.

2. Standard Test Cases

Test Case	Choice	n	Array Elements	Operation Input	Expected Output
TC1	1	5	5, 2, 8, 1, 4	Search key = 8	Sorted: 1 2 4 5 8; 8 found
TC2	1	5	5, 2, 8, 1, 4	Search key = 7	Sorted: 1 2 4 5 8; 7 not found
TC3	2	4	10, 20, 30, 40	Insert value = 25; Position = 3	After insertion: 10 20 25 30 40
TC4	2	5	10, 20, 30, 40, 50	Delete position = 3	After deletion: 10 20 40 50
TC5	3	5	1, 2, 3, 4, 5	k = 2	After rotation: 4 5 1 2 3
TC6	4	-	-	No additional input	Program exits

3. Algorithm

A. Common Menu Logic

1. Start.
2. Read the size n and the n array elements.
3. Display the menu options.
4. Read the user's choice.
5. Use switch-case to perform the selected operation.
6. Repeat the menu until choice 4 is selected.
7. Stop.

B. Bubble Sort and Binary Search

1. Sort the array in ascending order using Bubble Sort.
2. Display the sorted array.
3. Read the element to be searched.
4. Set low = 0 and high = n - 1.
5. While low <= high, calculate mid = (low + high) / 2.

6. If $a[\text{mid}]$ equals the search element, report its position and stop searching.
7. If the search element is smaller than $a[\text{mid}]$, set $\text{high} = \text{mid} - 1$; otherwise set $\text{low} = \text{mid} + 1$.
8. If the element is not found, display an appropriate message.

C. Insertion and Deletion

1. Display a submenu for insertion and deletion.
2. For insertion, read the element and a valid position from 1 to $n + 1$.
3. Shift elements one position to the right from the end up to the insertion position.
4. Store the new element and increase n by 1.
5. For deletion, read a valid position from 1 to n .
6. Shift all elements after that position one place to the left.
7. Decrease n by 1 and display the updated array.

D. Right Rotation by k Positions

1. Read k .
2. Reduce k using $k = k \% n$.
3. Repeat k times: store the last element, shift all remaining elements one place to the right, and place the stored element at index 0.
4. Display the rotated array.

4. C Program

```
#include <stdio.h>

int main()
{
    int a[100], n, i, j, choice;
    int temp, key, low, high, mid, found;
    int subChoice, value, position;
    int k, last;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    do
    {
        printf("\nMENU\n");
        printf("1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Rotate array right by k positions\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                for (i = 0; i < n - 1; i++)
```

```

{
    for (j = 0; j < n - i - 1; j++)
    {
        if (a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}

printf("Sorted array: ");
for (i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}

printf("\n");

printf("Enter element to search: ");
scanf("%d", &key);

low = 0;
high = n - 1;
found = 0;
while (low <= high)
{
    mid = (low + high) / 2;
    if (a[mid] == key)
    {
        printf("Element found at position %d\n", mid + 1);
    }
}

```

```
        found = 1;

        break;
    }
    else if (key < a[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

if (found == 0)
{
    printf("Element not found\n");
}

break;
```

case 2:

```
printf("1. Insertion\n");
printf("2. Deletion\n");
printf("Enter choice: ");
scanf("%d", &subChoice);
if (subChoice == 1)
{
    printf("Enter element to insert: ");
    scanf("%d", &value);
    printf("Enter position: ");
    scanf("%d", &position);
```

```

if (position >= 1 && position <= n + 1)
{
    for (i = n; i >= position; i--)
    {
        a[i] = a[i - 1];
    }
    a[position - 1] = value;
    n++;
    printf("Array after insertion: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}
else
{
    printf("Invalid position\n");
}
}

else if (subChoice == 2)
{
    printf("Enter position to delete: ");
    scanf("%d", &position);
    if (position >= 1 && position <= n)
    {
        for (i = position - 1; i < n - 1; i++)
        {

```

```

        a[i] = a[i + 1];
    }

    n--;

    printf("Array after deletion: ");

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n");
}

else
{
    printf("Invalid position\n");
}

}

else
{
    printf("Invalid choice\n");
}

break;

case 3:

    printf("Enter k: ");

    scanf("%d", &k);

    k = k % n;

    for (i = 0; i < k; i++)
    {
        last = a[n - 1];

        for (j = n - 1; j > 0; j--)

```

```
    {  
        a[j] = a[j - 1];  
    }  
    a[0] = last;  
}  
printf("Array after right rotation: ");  
for (i = 0; i < n; i++)  
{  
    printf("%d ", a[i]);  
}  
printf("\n");  
break;  
case 4:  
    printf("Exiting program\n");  
    break;  
default:  
    printf("Invalid choice\n");  
}  
}  
while (choice != 4);  
return 0;  
}
```


5. Compilation and Execution Evidence

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ ./question
Enter number of elements: 5
Enter array elements: 5
2
8
1
4

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 1
Sorted array: 1 2 4 5 8
Enter element to search: 8
Element found at position 5

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 1
Sorted array: 1 2 4 5 8
Enter element to search: 7
Element not found

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 4
Exiting program

(kali@kali)-[~/Desktop]
$ ./question
Enter number of elements: 5
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

Exiting program

(kali@kali)-[~/Desktop]
$ ./question
Enter number of elements: 5
Enter array elements: 10
20
30
40
50

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 2
1. Insertion
2. Deletion
Enter choice: 1
Enter element to insert: 25
Enter position: 3
Array after insertion: 10 20 25 30 40 50

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 2
1. Insertion
2. Deletion
Enter choice: 2
Enter position to delete: 3
Array after deletion: 10 20 30 40 50

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
```

```
Session Actions Edit View Help
Enter choice: 2
Enter position to delete: 3
Array after deletion: 10 20 30 40 50

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 4
Exiting program

(kali@kali)-[~/Desktop]
$ ./question
Enter number of elements: 5
Enter array elements: 1
2
3
4
5

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 3
Enter k: 2
Array after right rotation: 4 5 1 2 3

MENU
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 4
Exiting program

(kali@kali)-[~/Desktop]
$
```

6. Verification of All Test Cases

Test Case	Choice	n	Array Elements	Operation Input	Expected / Actual Result	Status
TC1	1	5	5, 2, 8, 1, 4	Search key = 8	Sorted: 1 2 4 5 8; 8 found	Pass
TC2	1	5	5, 2, 8, 1, 4	Search key = 7	Sorted: 1 2 4 5 8; 7 not found	Pass
TC3	2	4	10, 20, 30, 40	Insert value = 25; Position = 3	10 20 25 30 40	Pass
TC4	2	5	10, 20, 30, 40, 50	Delete position = 3	10 20 40 50	Pass
TC5	3	5	1, 2, 3, 4, 5	k = 2	4 5 1 2 3	Pass
TC6	4	-	-	No additional input	Program exits	Pass